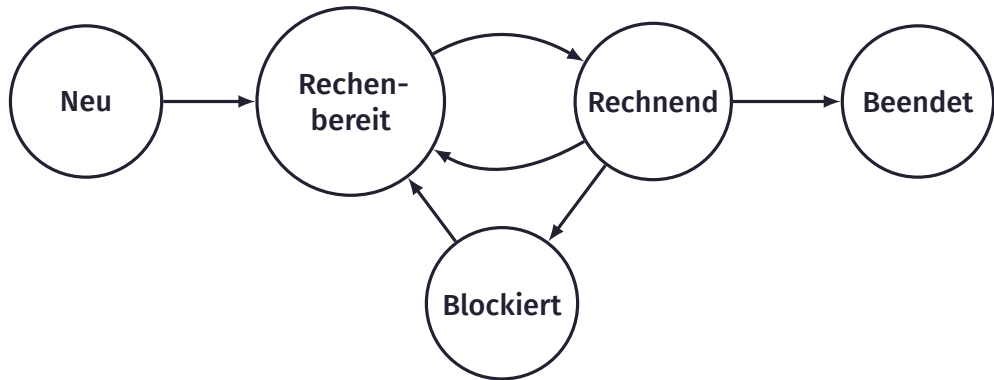


PROZESSZUSTÄNDE



PROZESSZUSTÄNDE

Beispielaufgabe:

- Das System hat 1 Festplatte und nutzt eine Warteschlange für Festplattenzugriffe (FCFS)
- Latenz für Festplattenzugriff: 30ms Start: 2 Prozesse A und B mit einer Laufzeit von jeweils 20ms
- Nachdem Prozess A 10ms lief, greift er auf die Festplatte zu
- Nachdem Prozess B 15ms lief, greift er auf die Festplatte zu

Hinweis: Falls 2 Prozesse simultan eintreffen, werden diese in die Warteschlange alphabetisch aufsteigend einsortiert.

PROZESSZUSTÄNDE - BEISPIELAUFGABE

A: Running (Active), R: Ready, B: Blocked, C: Completed

Zeit (ms)	Prozesse		Ready-Queue	HDD-Queue	Notizen
	A	B			
0	A	R	B		
10	B	A		A	
25	B	B		A,B	
40	A	B		B	A finishes HDD
50	C	B		B	
70	C	A			B finishes HDD
75	C	C			

PID

- **pid**: Identifikator eines Prozesses
`pid_t pid = getpid();`
- **ppid**: Identifikator des Elternprozesses
`pid_t ppid = getppid();`
- es existieren **nur Syscalls** für die PID und PPID

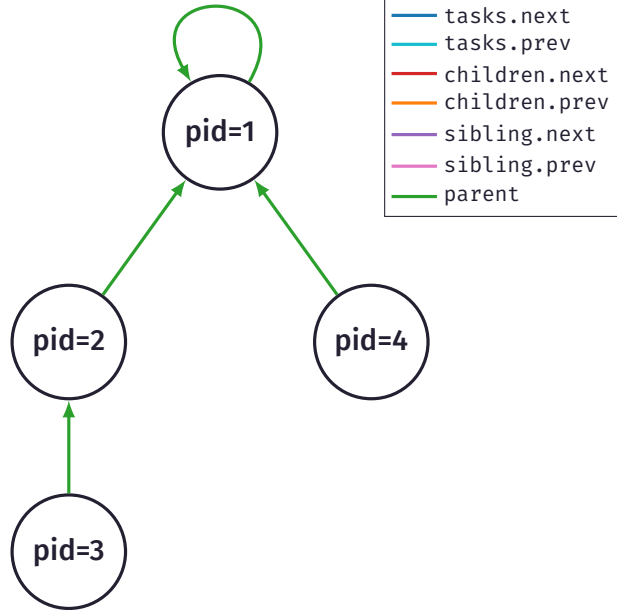
PROZESSBAUM

```
$ps -ef --forest
UID      PID      PPID  C  STIME TTY          TIME CMD
mschroe+ 144583        1  0 14:35 ?        00:00:00 /usr/bin/zsh
mschroe+ 144593    144583  0 14:35 ?        00:00:00 \_ xterm
mschroe+ 144595    144593  1 14:35 pts/7    00:00:03 \_ zsh
mschroe+ 145281    144595  9 14:37 pts/7    00:00:06 \_ /usr/lib/chromium/chromium google.com
mschroe+ 145290    145281  0 14:37 pts/7    00:00:00 \_ \_ /usr/lib/chromium/chromium
mschroe+ 145315    145290  1 14:37 pts/7    00:00:01 \_ \_ \_ /usr/lib/chromium/chromium
mschroe+ 145291    145281  0 14:37 pts/7    00:00:00 \_ \_ /usr/lib/chromium/chromium
mschroe+ 145293    145291  0 14:37 pts/7    00:00:00 \_ \_ \_ /usr/lib/chromium/chromium
mschroe+ 145321    145293  0 14:37 pts/7    00:00:00 \_ \_ \_ \_ /usr/lib/chromium/chromium
mschroe+ 145346    145293  4 14:37 pts/7    00:00:02 \_ \_ \_ \_ /usr/lib/chromium/chromium
mschroe+ 145318    145281  1 14:37 pts/7    00:00:00 \_ \_ /usr/lib/chromium/chromium
mschroe+ 145665    145281  0 14:37 pts/7    00:00:00 \_ \_ /usr/lib/chromium/chromium
mschroe+ 146017    144595 10 14:37 pts/7    00:00:05 \_ /usr/lib/firefox/firefox /tmp/callbacks.html
mschroe+ 146103    146017  0 14:37 pts/7    00:00:00 \_ \_ /usr/lib/firefox/firefox
mschroe+ 146122    146017  0 14:37 pts/7    00:00:00 \_ \_ /usr/lib/firefox/firefox
mschroe+ 146271    146017  0 14:37 pts/7    00:00:00 \_ \_ /usr/lib/firefox/firefox
mschroe+ 146317    144595  0 14:38 pts/7    00:00:00 \_ nvim test.c
mschroe+ 146320    146317  4 14:38 ?        00:00:01 \_ \_ nvim --embed test.c
mschroe+ 146335    146320  8 14:38 ?        00:00:02 \_ \_ node
↪ /home/mschroetter/.local/share/nvim/lazy/copilot.lua/copilot/index.js$
```

→ Prozessbeziehungen sind “tree-like”

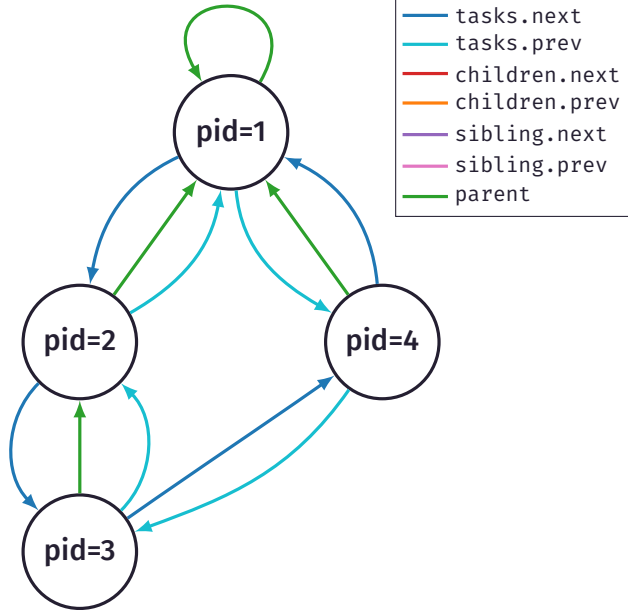
LINUX PROZESSTABELLE

```
struct task_struct {  
    ...  
    struct list_head tasks;  
    pid_t pid;  
    struct task_struct *parent;  
    struct list_head children;  
    struct list_head sibling;  
    ...  
}  
  
struct list_head {  
    struct list_head *next, *prev;  
};
```



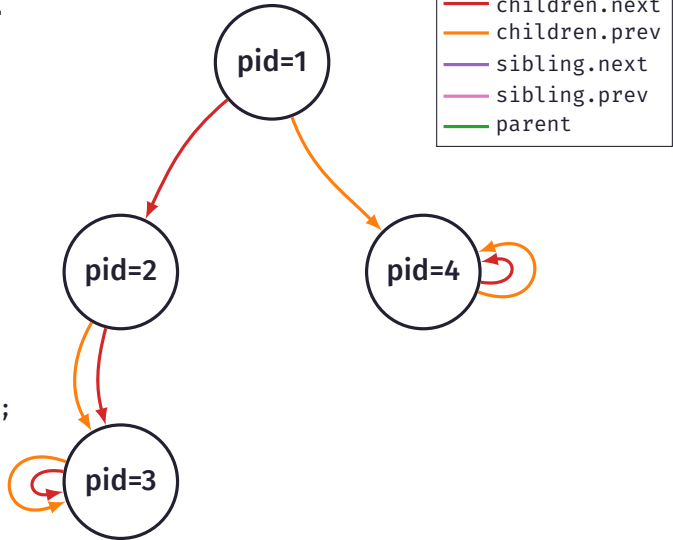
LINUX PROZESSTABELLE

```
struct task_struct {  
    ...  
    struct list_head tasks;  
    pid_t pid;  
    struct task_struct *parent;  
    struct list_head children;  
    struct list_head sibling;  
    ...  
}  
  
struct list_head {  
    struct list_head *next, *prev;  
};
```



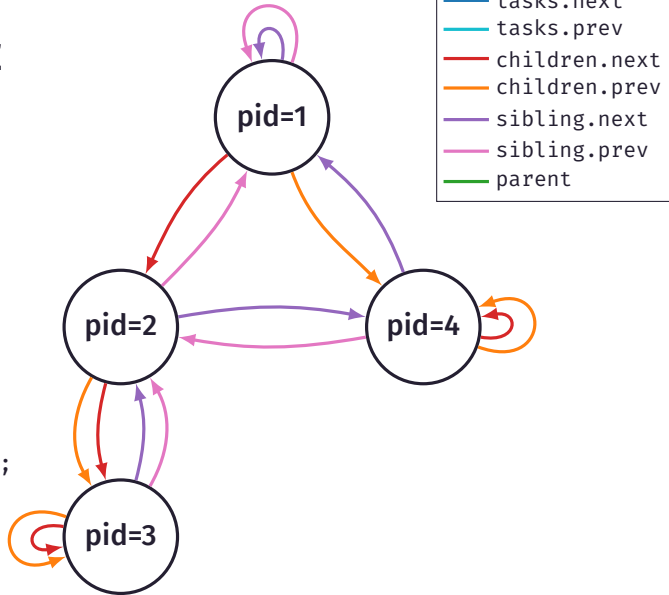
LINUX PROZESSTABELLE

```
struct task_struct {  
    ...  
    struct list_head tasks;  
    pid_t pid;  
    struct task_struct *parent;  
    struct list_head children;  
    struct list_head sibling;  
    ...  
}  
  
struct list_head {  
    struct list_head *next, *prev;  
};
```



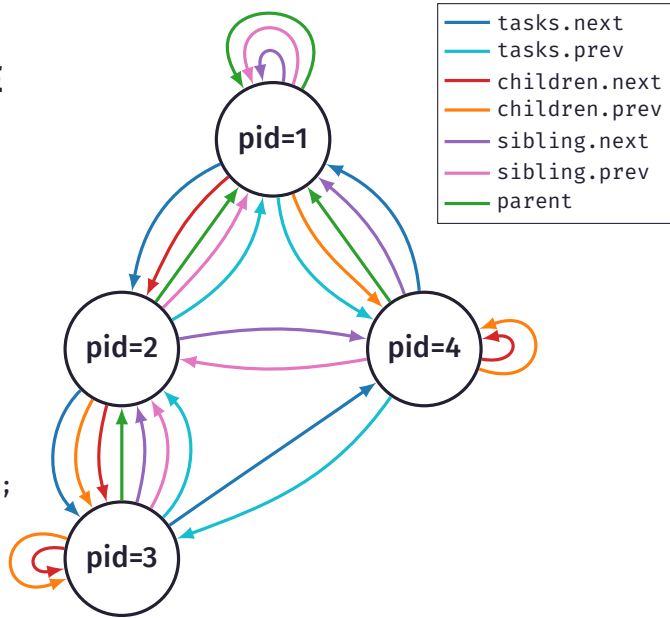
LINUX PROZESSTABELLE

```
struct task_struct {  
    ...  
    struct list_head tasks;  
    pid_t pid;  
    struct task_struct *parent;  
    struct list_head children;  
    struct list_head sibling;  
    ...  
}  
  
struct list_head {  
    struct list_head *next, *prev;  
};
```



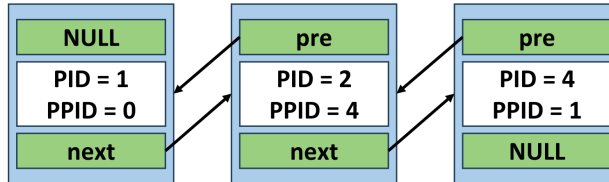
LINUX PROZESSTABELLE

```
struct task_struct {  
    ...  
    struct list_head tasks;  
    pid_t pid;  
    struct task_struct *parent;  
    struct list_head children;  
    struct list_head sibling;  
    ...  
}  
  
struct list_head {  
    struct list_head *next, *prev;  
};
```

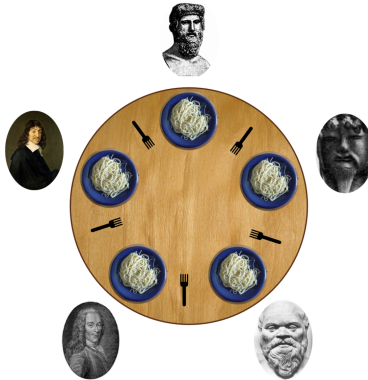


AUFGABE PROZESSTABELLE

- Implementierung einer vereinfachten Prozesstabelle in C
- Ziel: Wiederholung C-Programmierung & Verstehen von bekannten Kernel Datenstrukturen
- Datenstruktur: ist euch überlassen (**mindestens doppelt verkettete Liste**)
- Eigenschaften wie Kind-Beziehung müssen nicht im Datentyp abgebildet werden, können auch über Funktionen realisiert werden (itterieren über alle Prozesse etc.)

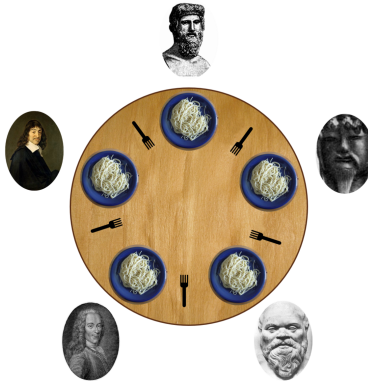


ABNORMALE ZUSTÄNDE



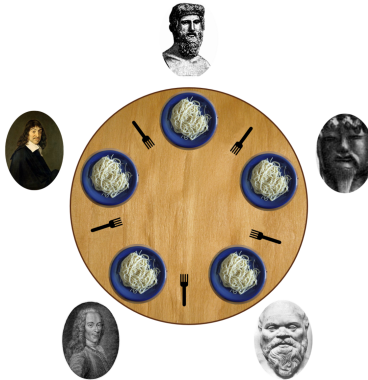
Deadlock: alle Phil. greifen sich das rechte Stäbchen und **warten** darauf, dass das linke Stäbchen frei wird.

ABNORMALE ZUSTÄNDE



Livelock: alle Phil. greifen sich das rechte Stäbchen, wenn das Linke nicht vorhanden ist, legen sie es wieder hin und **versuchen es erneut**.

ABNORMALE ZUSTÄNDE



Starvation: Phil. prüft ständig, ob beide Stäbchen vorhanden sind. Sind beide Stäbchen für einen Phil. vorhanden, dann greift er beide und isst. **Ein Phil. wird verhungern**, wenn zu jedem Zeitpunkt mindestens einer seiner Nachbarn isst.

PROZESSZUSTÄNDE

PROCESS STATE CODES

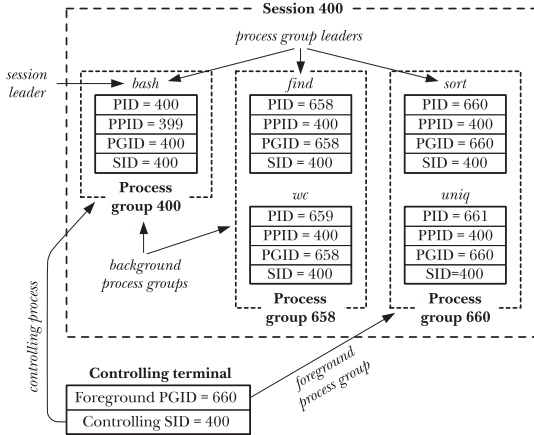
Here are the different values that the s, stat and state output specifiers (header "STAT" or "S") will display to describe the state of a process:

```
D    uninterruptible sleep (usually I/O)
I    idle kernel thread
R    running or runnable (on run queue)
S    interruptible sleep (waiting for an event to complete)
T    stopped by job control signal
t    stopped by debugger during the tracing
W    paging (not valid since Linux 2.6)
X    dead (should never be seen)
Z    defunct ("zombie") process, terminated but not reaped by its parent
```

For BSD formats and when the `stat` keyword is used, additional characters may be displayed:

```
< high-priority (not nice to other users)
N low-priority (nice to other users)
L has pages locked into memory (for real-time and custom I/O)
S is a session leader
l is multi-threaded (using CLONE_THREAD, like NPTL pthreads do)
+ is in the foreground process group
```

JOB CONTROL



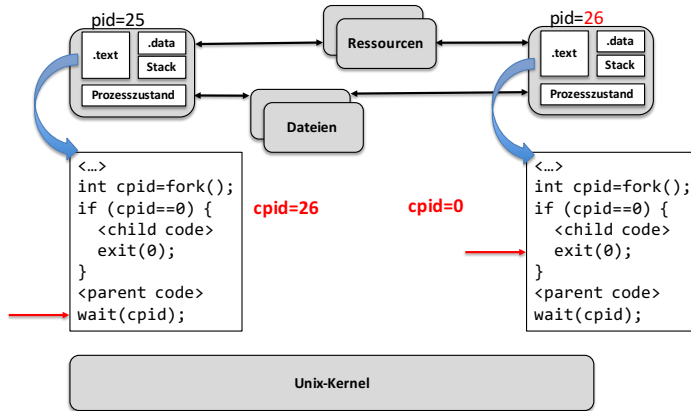
\$ echo \$\$

400

```
$ find / 2> /dev/null | wc -l &
[1] 659
```

```
$ sort < longlist | uniq -c
```

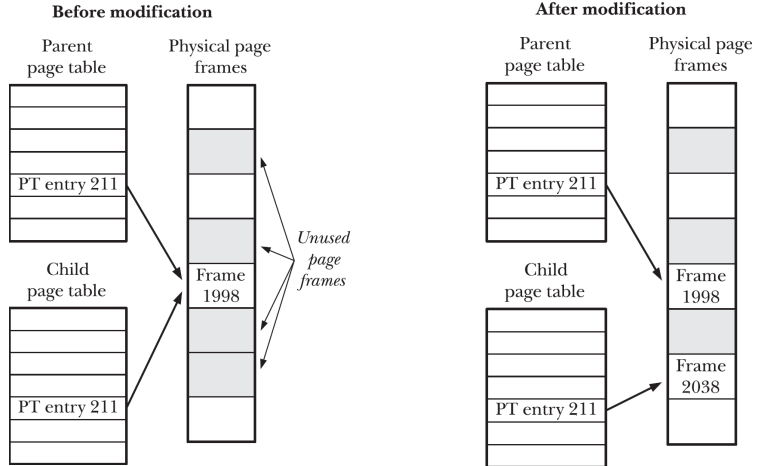

ABLAUF VON FORK



`pid25` wartet auf Ende von `pid26`, **`pid26`** führt `exit(0)` aus und beendet sich

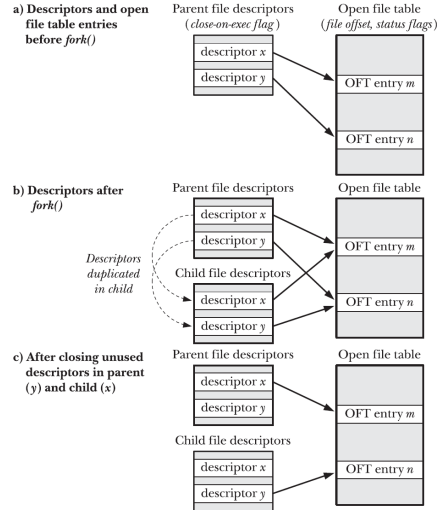
MEMORY AFTER FORK

- `fork()` kopiert den **Adressraum** des Elternprozesses
- Copy-on-Write Optimierung **reduziert** Prozess-erzeugungszeit und Speicherverbrauch
- Adressraum ist kopiert **nach Modifizierung**

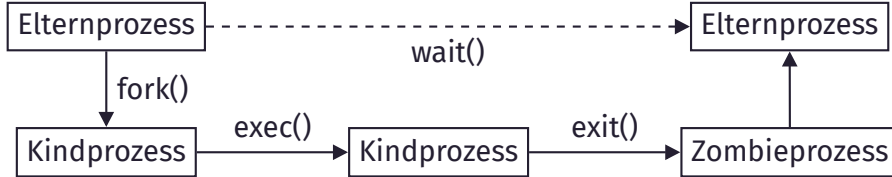


FILES AFTER FORK

- `fork()` kopiert **alle** offenen File-Descriptors (Dateireferenzen)
- inkludiert **Datei-Offset**
- schließen eines File-Descriptors im Kind hat **keine Auswirkung** auf den Elternprozess (und andersherum)



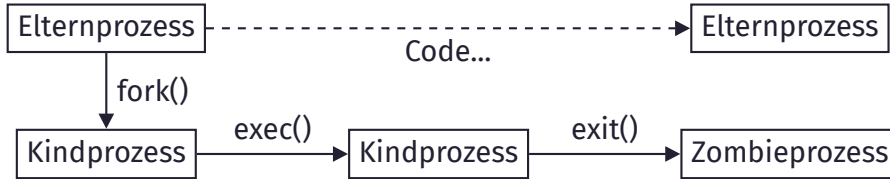
PROZESSZYKLUS



Funktion von **exec()**:

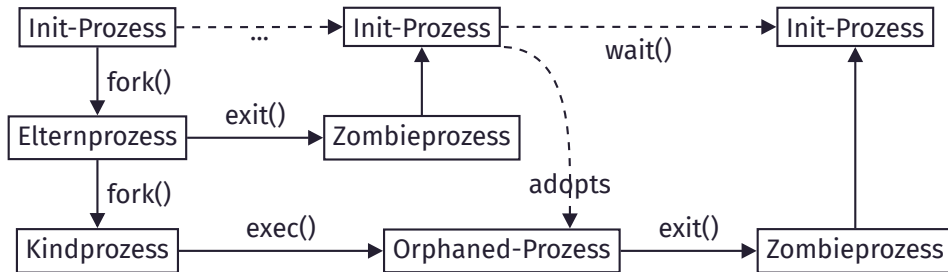
- **ersetzen des Speicherinhaltes** des aktuellen Prozesses durch Sektionen anderer ausführbarer Datei (Parameter von exec)
- Methode **neue Programme** zu starten, denn fork() erstellt **nur Kopie** des aufgerufenen Prozesses

ZOMBIES



- wenn **wait()** oder **waitpid()** vom Elternprozess nicht aufgerufen wird, verweilen Kinder in einem sogenannten „**Zombie**“-Zustand
- **wait()** muss aufgerufen werden, um die Ressourcen des Kindprozesses freizugeben

ORPHANED-PROZESSE



- Orphaned-Prozesse werden vom Init-Prozess „adoptiert“
- Prozessergebnisse/Exit-Codes können **nicht evaluiert** werden (Fehler können nicht erkannt werden)
- **Eltern sollten immer auf ihre Kinder warten!**
- aufrufen bei unserem Beispiel mithilfe von **wait(NULL)**, denn Informationen sind nicht relevant → siehe **man wait**

SCHEDULING ALGORITHMEN

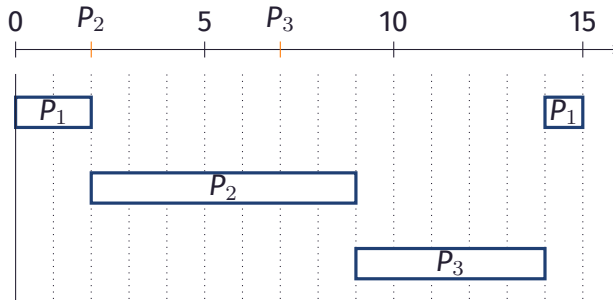
In einem Rechensystem liegen die folgenden Tasks vor:

Prozess	Ankunftszeit	Rechenzeit	Priorität
P_1	0	3	1
P_2	2	7	3
P_3	7	5	2

1. Erstellen Sie ein Gantt-Diagramm für einen **preemptive** Scheduling-Algorithmus mit *fixed priorities*.
2. Berechnen Sie die **mittlere Wartezeit**, die **mittlere Verweildauer** sowie die **mittlere Antwortzeit**.

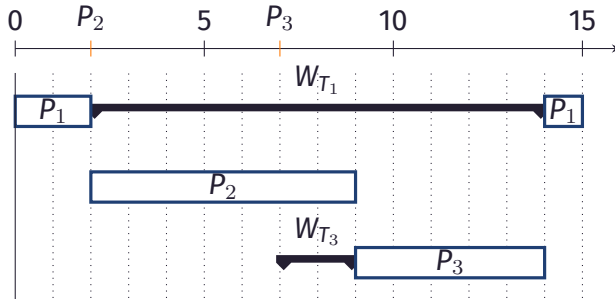
GANTT-DIAGRAMM

Prozess	Ankunftszeit	Rechenzeit	Priorität
P_1	0	3	1
P_2	2	7	3
P_3	7	5	2



MITTLERE WARTEZEIT

Prozess	Ankunftszeit	Rechenzeit	Priorität
P_1	0	3	1
P_2	2	7	3
P_3	7	5	2

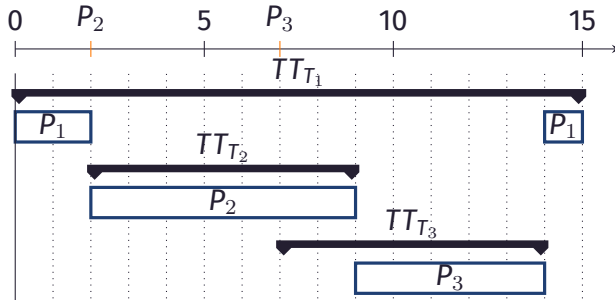


$$\begin{aligned} W_T &= \frac{W_{T_1} + W_{T_2} + W_{T_3}}{3} \\ &= \frac{12 + 0 + 2}{3} \\ &= \frac{14}{3} \end{aligned}$$

Welche Formel kann für die Wartezeit W_{T_i} angegeben werden?

MITTLERE VERWEILDAUER

Prozess	Ankunftszeit	Rechenzeit	Priorität
P_1	0	3	1
P_2	2	7	3
P_3	7	5	2

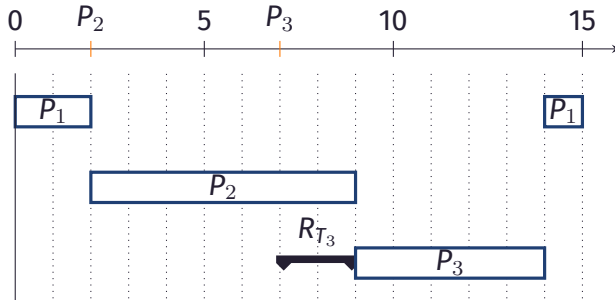


$$\begin{aligned}
 T_T &= \frac{TT_{T_1} + TT_{T_2} + TT_{T_3}}{3} \\
 &= \frac{15 + 7 + 7}{3} \\
 &= \frac{29}{3}
 \end{aligned}$$

Welche Formel kann für die Wartezeit T_{T_i} angegeben werden?

MITTLERE ANTWORTZEIT

Prozess	Ankunftszeit	Rechenzeit	Priorität
P_1	0	3	1
P_2	2	7	3
P_3	7	5	2



$$\begin{aligned} R_T &= \frac{R_{T_1} + R_{T_2} + R_{T_3}}{3} \\ &= \frac{0 + 0 + 2}{3} \\ &= \frac{2}{3} \end{aligned}$$

Welche Formel kann für die Wartezeit R_{T_i} angegeben werden?

FIFO

```
int schedule_fifo(process_t *current_process, process_t
↪ **next_process) {
    if(current_process && current_process->state == READY) {
        *next_process = current_process;
        return 0;
    }

    if (queue_length(current_core->run_queue) < 1) {
        return -1;
    }

    return queue_dequeue(current_core->run_queue,
↪ next_process);
}
```

AUFGABE 3: WINDOWS-SCHEDULER

- nutze modifiziertes Gantt-Diagramm: **y-Achse = Priorität**
- **3 Szenarien** für Prozess-Scheduling:
 1. Prozess war **blockiert (Netzwerk-I/O)** und erhält nun die CPU
 2. Prozess ist **GUI-Thread**
 3. Prozess hat schon seit **mehr als 4 Sekunden** nicht mehr gerechnet

Zusatzinformation aus Windows Internals Seventh Edition Part 1:

*On client version of Windows, threads run for **two clock intervals by default**. [...] Internally, a quantum unit is represented as **one-third of a clock tick**. That is, one clock tick equals three quanta. This means that on client Windows systems, threads have a **quantum reset value of 6** ($2 * 3$) [...]*

Hinweis: Gehen Sie von einem Clock-Interval von 10ms aus!

AUFGABE 3: WINDOWS-SCHEDULER

- Einem Thread sind eine **baseprio** und eine **current_prio** zugeordnet:

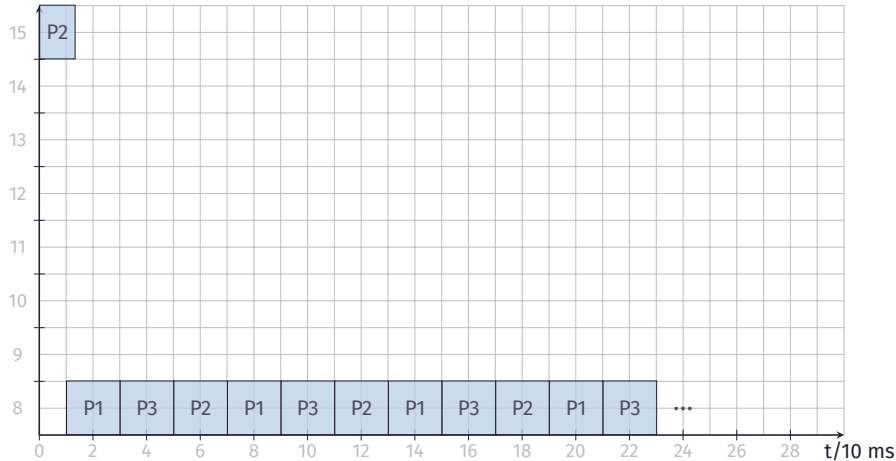
$$current_prio_neu := \min\{baseprio + boost; 15\}$$

Standard: baseprio = 8

- **Priorität und Quantum** eines Threads werden in gewissen Situationen kurzfristig verändert, z.B.:
 1. **nach I/O (Network):** $boost = 2$ Nach jeder Zeitscheibe wird die Priorität (**current_prio**) um 1 gesenkt, bis die **baseprio** wieder erreicht ist.
 2. **Threads einer interaktiven Sitzung (z.B. GUI-Threads):** $boost = 6$, gleichzeitig wird die Anzahl der Quantum-Units verdreifacht
 3. **Threads, die länger als 4 s im Zustand rechenbereit sind:**
 $current_prio_neu := 15$, gleichzeitig wird Anzahl der Quantum-Units auf 3 reduziert (→ **Balance Set Manager**)
- ! **Achtung:** In Fall 2 und 3 verfällt der Boost sofort nach Ablauf eines Quantums!

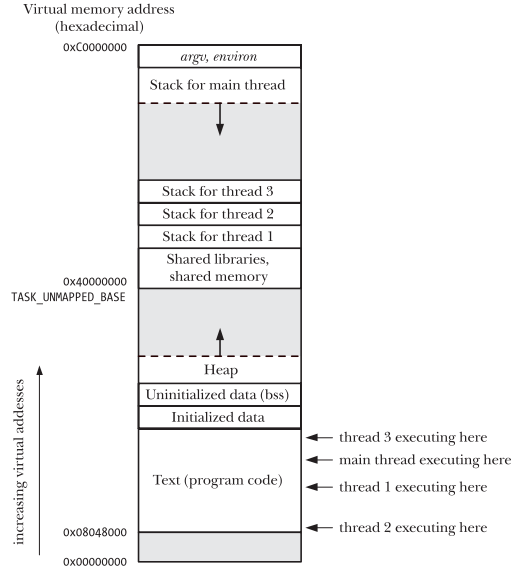
WINDOWS SCHEDULER c)

Im dritten Szenario hat Prozess 2 schon seit mehr als 4 Sekunden nicht gerechnet.



PTHREAD API

- `pthread_attr_init` initialisiert das `pthread_attr_t` struct
- `pthread_attr_...` set und get Attribute eines Threads
- `pthread_create` erstellt einen neuen Thread
- `pthread_join` wartet auf die Beendigung des angegebenen Threads
- `pthread_exit` terminiert den aktuellen Thread
- `pthread_detach` "detaches" diesen Thread
- ...



PTHREAD_CREATE

```
int pthread_create(pthread_t *restrict thread, // the thread handle
                  const pthread_attr_t *restrict attr, // attributes can be NULL
                  ↪ to use defaults
                  void *(*start_routine)(void *), // the function to be executed
                  void *restrict arg); // void pointer to the arguments of the
                  ↪ function
```

- Argumente: falls Sie **mehr als ein Argument** übergeben wollen, übergeben Sie einen **pointer** zu einer Struktur bestehend aus den Argumenten.
- Für **jeden** erfolgreich erzeugten thread, **muss pthread_join()** aufgerufen werden,
- **außer** wenn **pthread_detach()** aufgerufen wurde oder **PTHREAD_CREATE_DETACHED** in den Attributen gesetzt wurde.
- Treffen Sie **keine Annahmen über das Scheduling**.

PROZESSE & THREADS IN LINUX

- Der Linux Kernel unterscheidet nicht zwischen Prozessen und Threads, alles sind **Tasks**.
- Sie unterscheiden sich nur durch ihre Eigenschaften wie **Adressraum** und **Ressourcen**.
- Wenn ein Programm in Linux gestartet wird, wird ein **Task** im Kernel erzeugt.
- Wenn dieser Task weitere Threads erzeugt sind dies ebenfalls Tasks.
- Diese unterscheiden sich nur durch ihren Stack und Attributen.
- Sobald ein Task `exit` (glibc) aufruft, werden alle Tasks der selben PID beendet.
- Zum beenden eines einzelnen Threads innerhalb eines Prozesses muss `pthread_exit` aufgerufen werden.
- Das Joinen nicht detachter Threads ist notwendig, um **Resource-Leaks** zu vermeiden.

VRUNTIME

- CFS führt die Scheduling Entscheidungen basierend auf der **virtual runtime** einer Task aus

$$\text{vruntime} = \text{delta_exec} \times \frac{\text{NICE_0_LOAD}}{\text{sched_prio_to_weight}(\text{prio})}$$

- CFS sortiert die “Tasks” als Scheduling-Entitäten in einem **Red-Black Tree (RBT)**
- **jeder Thread** gilt als eine andere Entität
→ Besonderheit von **Linux**: je mehr Threads ein Prozess hat, umso **mehr CPU-Zeit** bekommt er

PRIORITIES

Nice levels:

Nice levels are **multiplicative**, with a gentle **10% change** for **every nice level changed**. I.e. when a CPU-bound task goes from nice 0 to nice 1, it will get 10% less CPU time than another CPU-bound task that remained on nice 0. [...] to achieve that we use a **multiplier of 1.25**.

```
const int sched_prio_to_weight[40] = {  
    /* -20 */      88761,      71755,      56483,      46273,      36291,  
    /* -15 */      29154,      23254,      18705,      14949,      11916,  
    /* -10 */      9548,       7620,       6100,       4904,       3906,  
    /*  -5 */      3121,       2501,       1991,       1586,       1277,  
    /*   0 */      1024,        820,        655,        526,        423,  
    /*   5 */       335,        272,        215,        172,        137,  
    /*  10 */       110,         87,         70,         56,         45,  
    /*  15 */        36,         29,         23,         18,         15,  
};
```

SCHEDULING GROUPS

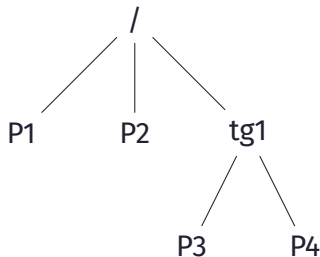
Group Scheduler Extensions to CFS:

Normally, the scheduler operates on individual tasks and strives to provide fair CPU time to **each task**. **Sometimes**, it may be desirable to group tasks and provide fair CPU time to **each such task group**. For example, it may be desirable to first provide fair CPU time to each user on the system and then to each task belonging to a user.

- **Gruppen** werden ebenfalls als Scheduling-Entitäten betrachtet
- default prio für Tasks und Gruppen beträgt 1024 (**entspricht NICE_0_LOAD**)

SCHEDULING GROUPS

- **jede Gruppe** hat zur Verwaltung ihren **eigenen RBT**, Beispiel:
 - Run queue (/) : P1, P2, tg1
 - Run queue (tg1): P3, P4



UPDATING VRUNTIME

- **update_curr** aktualisiert die vruntime des **derzeitigen** Tasks **und** aller ihrer "Vorfahren"
- der RBT wird **balanciert**, wenn die vruntime **aktualisiert oder** eine Task **hinzugefügt** wird
- **Unterschied** bei einem idealen System: CPU-Zeit ist gleich dem **Quotienten** bestehend aus der load des einzelnen Tasks durch die load der gesamten Run-Queue

RED-BLACK-TREE

Grundlegende Eigenschaften:

1. Wurzel ist **schwarz**
2. Kinder von **roten** Knoten sind **schwarz**
3. gleiche **Schwarzhöhe**: **jeder Pfad** ausgehend von **einem gegebenen Knoten** bis zu einem Blatt enthält **dieselbe Anzahl** an schwarzen Knoten

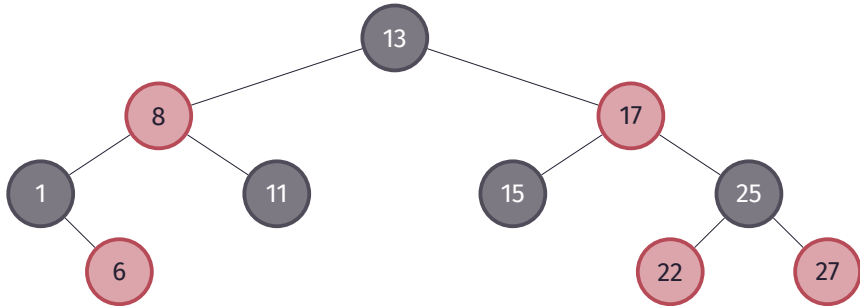
RED-BLACK-TREE

Operation - Einfügen:

1. Knoten wie in **binären Suchbaum** mit roter Farbe einfügen
2. Fall: **Elternteil schwarz**: Fertig
3. Fall: **Elternteil rot**, dann rebalancieren:
 - Fall: Onkel (von Elternteil) ist **schwarz**:
 - ▶ der neue Knoten ist das äußere Kind →rotiere **Großelternteil** (bspw. LL-Rotation)
 - ▶ der neue Knoten ist das innere Kind →rotiere **Großelternteil und Elternteil** (bspw. LR-Rotation)
 - ▶ **färbe** Großelternteil und Elternteil passend
 - Fall: Onkel (von Elternteil) ist **rot**: →färbe Elternteil und Onkel **schwarz** sowie Großelternteil **rot**, falls es nicht die Wurzel ist

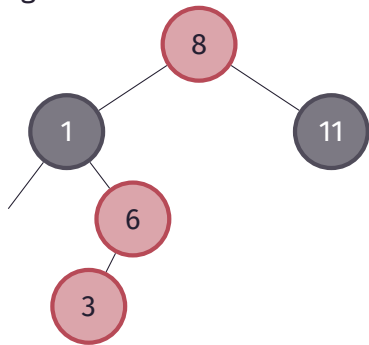
RED-BLACK-TREE

Betrachte das folgende Beispiel:



REBALANCING RED-BLACK TREE (I)

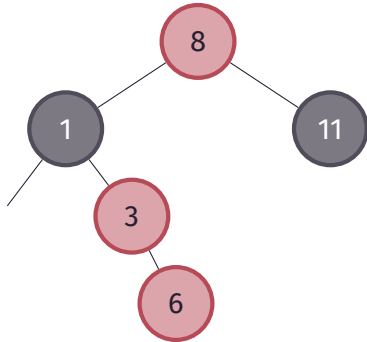
1. Füge Task mit **vruntime=3** ein:



→Rebalanciere →Onkel (von Elternteil) ist schwarz →es ist ein inneres Kind
→**rotiere** Elternteil und Großelternteil (**RL-Rotation**)

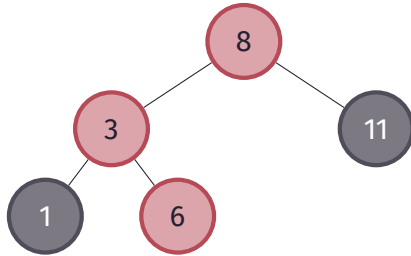
REBALANCING RED-BLACK TREE (II)

2.



REBALANCING RED-BLACK TREE (III)

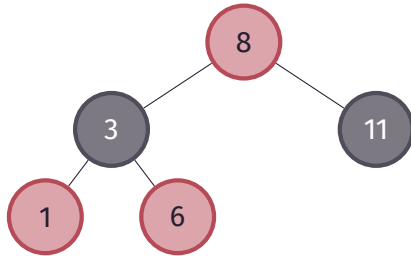
3.



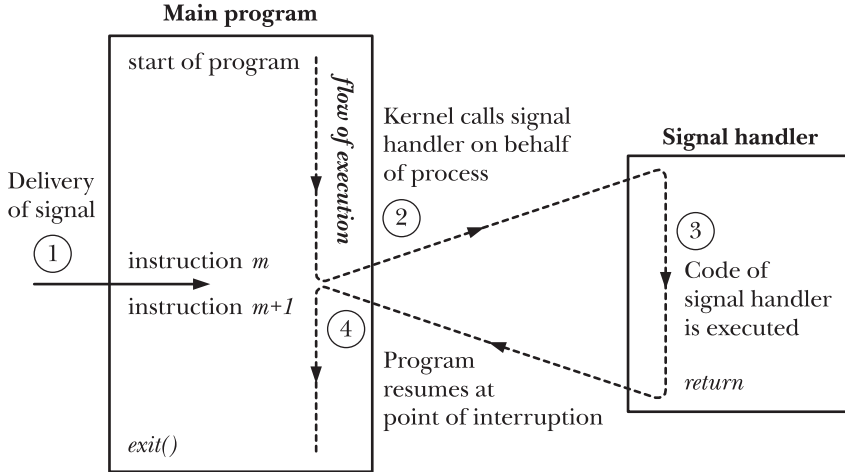
→ färbe Knoten, Elternteil und Großelternteil passend

REBALANCING RED-BLACK TREE (IV)

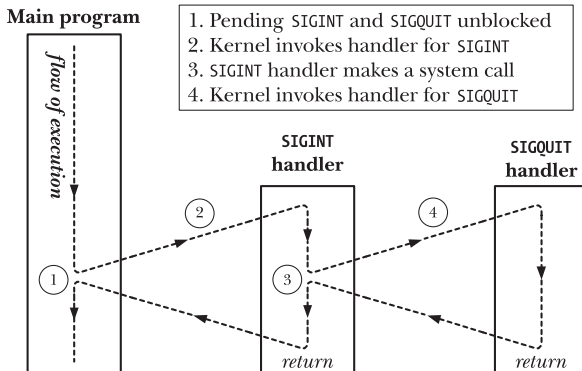
4.



SIGNALS



MULTIPLE SIGNALS



Mögliche Race Conditions:

- **Nur Async-Signal-Safe Funktionen** dürfen in Signalhandler aufgerufen werden! siehe: **man 7 signal-safety!**

SIGNALBEHANDLUNG

- jedes Signal hat eine **default Aktion**, häufig Terminierung des Prozesses (siehe **man 7 signal**)
- mittels **sigaction()** kann ein Signalhandler **registriert** werden oder ein Signal ignoriert werden
- mittels **sigprocmask()** kann ein Signal **temporär blockiert** werden
wird im Prozessleitblock (task_struct) gespeichert
- ein Signalhandler sollte **möglichst schnell** abgearbeitet werden
→ setzen einer sig_atomic_t Variable, die von der Hauptlogik des Programms ausgewertet wird

SIGNALHANDLER FÜR BROKEN PIPE

- Signal-Action mit **sigaction** registrieren

```
#include <signal.h>
```

```
int sigaction(int signum, const struct sigaction *act,  
              struct sigaction *oldact);
```

- **struct sigaction** beschreibt die Action:

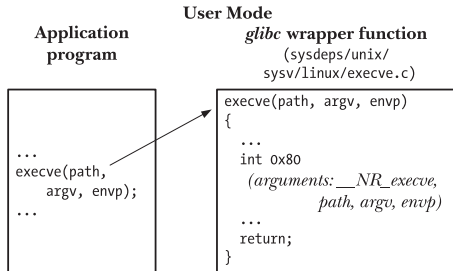
```
struct sigaction {  
    void (*sa_handler)(int);  
    void (*sa_sigaction)(int, siginfo_t *, void *);  
    sigset_t sa_mask;  
    int sa_flags;  
    void (*sa_restorer)(void);  
};
```

- **void (*sa_handler)(int)**: Funktion, die Signal behandelt
- **sigset_t sa_mask**: Menge anderer zu blockierender Signale setzen mit **sigemptyset**, **sigaddset**,... (→ siehe: **man SIGSETOPS**)
- **int sa_flags**: Flags, die das Verhalten des Signals verändern

(Nachlesen & Beispiel unter: **man 2 sigaction**)

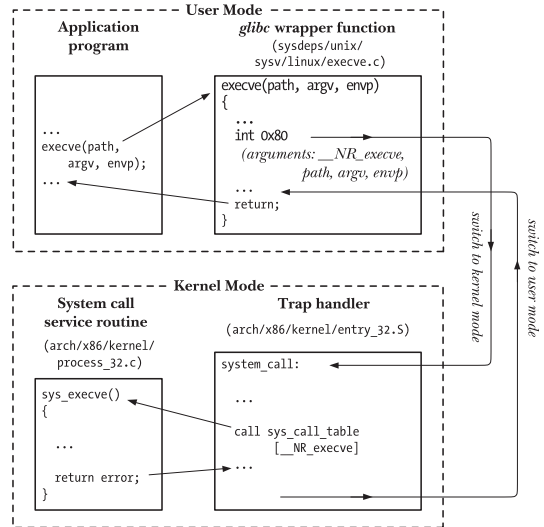
SYSCALLS - 32 BIT LINUX

- Aufrufe wie **fork()** oder **execve()** sind **Wrapperfunktionen** in der Glibc
- **Wrapperfunktionen** = Funktionen, die andere Programmkomponenten „umgeben“ (in diesem Falle aus Kernel Mode)
- aufrufendes Programm steht im Austausch mit **Wrapper**



SYSCALLS - 32 BIT LINUX

- Glibc führt **Interrupt 0x80** aus um in den Kernel-Mode zu wechseln
- X86_64: **syscall** Instruction
- **Argumente** des Syscalls müssen in **Registern** gesetzt werden
- Syscalls können auch **direkt** über den **syscall()-Wrapper** aufgerufen werden
- **syscall** wird via **Nummer im ersten Argument** spezifiziert



SYSCALL WRAPPER

Syntax:

```
long syscall(long number, ... );  
//syscall(SYS_befehl,argument_1,...);
```

```
#include <unistd.h>  
#include <stdio.h>  
#include <sys/syscall.h>
```

```
int main(int argc, char**argv){  
    int c;  
    c = syscall(SYS_getpid);  
    printf("%d", c);  
}
```

%rax	System call	%rdi	%rsi	%rdx
0	sys_read	unsigned int fd	char *buf	size_t count
1	sys_write	unsigned int fd	const char *buf	size_t count
2	sys_open	const char *filename	int flags	int mode

http://blog.rchapman.org/posts/Linux_System_Call_Table_for_x86_64/

GETTIMEOFDAY

SYNOPSIS

```
#include <sys/time.h>

int gettimeofday(struct timeval* tv, struct timezone* tz);
```

DESCRIPTION

The functions `gettimeofday()` and `settimeofday()` can get and set the time as well as a timezone.

The `tv` argument is a `struct timeval` (as specified in `<sys/time.h>`):

```
struct timeval {
    time_t      tv_sec;      /* seconds */
    suseconds_t tv_usec;     /* microseconds */
};
```

Quelle: `man gettimeofday`

HINWEISE

- Zeitmessung mit **time**
 - time ist ein Unix/Linux Programm und ein **bash-Built-In**
 - misst die Zeit, die ein Programm benötigt und schlüsselt diese nach **Userspace** und **Kernelspace** auf

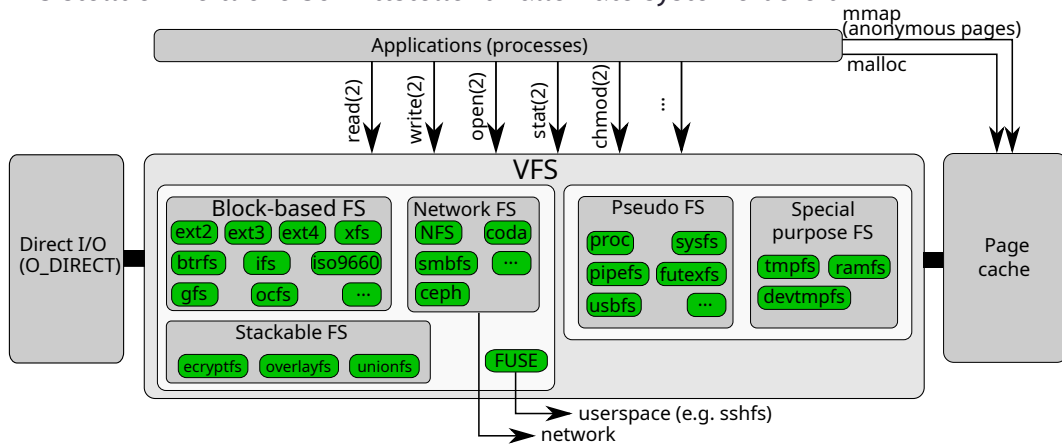
```
maxschro@tiree:~$ time ./a.out  
positive
```

```
real    0m0.002s  
user    0m0.002s  
sys     0m0.000s
```

- keine Abgabe via Gitlab sondern **im PDF**.

VIRTUAL FILE SYSTEM

- VFS stellt einheitliche Schnittstelle für alle Dateisysteme bereit



- VFS stellt Dateisysteme als Baumstruktur dar

AUFGABE 2: DAS PROC-DATEISYSTEM

Das **/proc**-Dateisystem bietet eine Schnittstelle

- zum Auslesen von Kernel-Informationen
- zum Ändern von Betriebssystemparametern (siehe **man sysctl** – `configure kernel parameters at runtime`)

/proc enthält

- Dateien und Dateiverzeichnisse mit Betriebssysteminformation wie z.B. `cpuinfo`, `interrupts`, `partitions`,...
- für jeden Prozess ein Unterverzeichnis, das nach der zugehörigen PID benannt ist
 - darin Informationen zum Prozess, z.B. **stat** (Status-Informationen), **maps** (aktuell gemappte Speicherregionen), **fd** (Datei-Deskriptoren der offenen Dateien),...

AGENDA

1. Syscalls
2. VFS & Inode
3. Hinweise Blatt 4

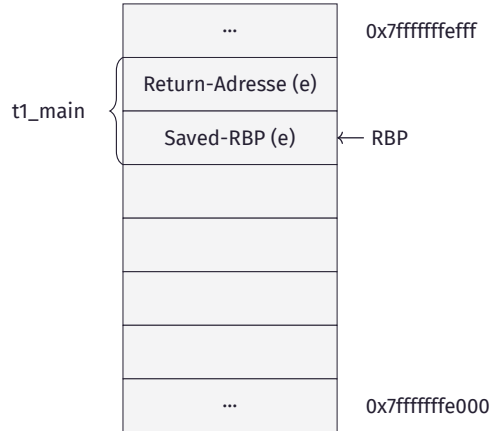


STACKFRAMES

```
void t1_f1(void){
    int x=3;
    p=&x;
    sleep(1);
}
```

```
void t1_f2(void){
    int y=5;
    k++;
    sleep(1);
}
```

```
void* t1_main(void* args){
    t1_f1();
    t1_f2();
    return NULL;
}
```

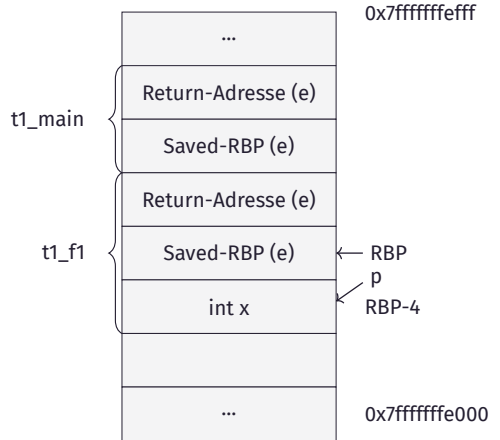


STACKFRAMES

```
void t1_f1(void){
    int x=3;
    p=&x;
    sleep(1);
}
```

```
void t1_f2(void){
    int y=5;
    k++;
    sleep(1);
}
```

```
void* t1_main(void* args){
    t1_f1();
    t1_f2();
    return NULL;
}
```

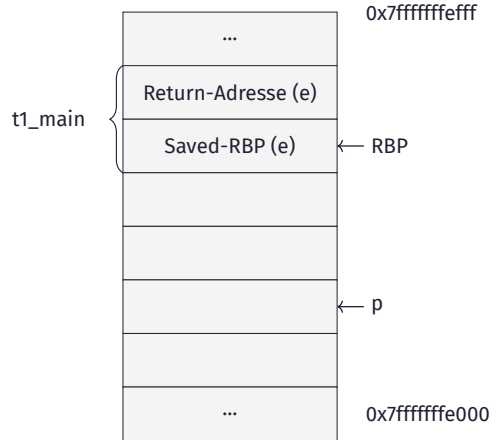


STACKFRAMES

```
void t1_f1(void){  
    int x=3;  
    p=&x;  
    sleep(1);  
}
```

```
void t1_f2(void){  
    int y=5;  
    k++;  
    sleep(1);  
}
```

```
void* t1_main(void* args){  
    t1_f1();  
    t1_f2();  
    return NULL;  
}
```

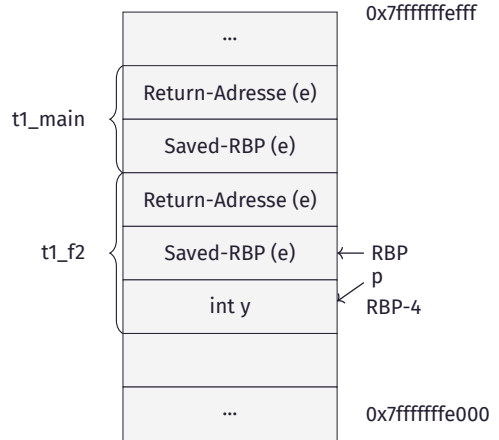


STACKFRAMES

```
void t1_f1(void){  
    int x=3;  
    p=&x;  
    sleep(1);  
}
```

```
void t1_f2(void){  
    int y=5;  
    k++;  
    sleep(1);  
}
```

```
void* t1_main(void* args){  
    t1_f1();  
    t1_f2();  
    return NULL;  
}
```



SYNCHRONISATION NEBENLÄUFIGER PROZESSE

Reader:

```
(...)  
start_read();  
read(Daten);  
end_read();  
(...)
```

Writer:

```
(...)  
start_write();  
write(Daten);  
end_write();  
(...)
```

Initialisierung gemeinsamer Variablen:

```
readcount = 0; /* Zaehler fuer Lesewuensche */  
writecount = 0; /* Zaehler fuer Schreibwuensche */  
mutex_rc = 1; /* bin. Semaphor fuer exkl. Zugriff auf readcount */  
mutex_wc = 1; /* bin. Semaphor fuer exkl. Zugriff auf writecount */  
mutex_str = 1; /* bin. Semaphor fuer exkl. Zugriff auf start_read */  
  
sem_w = 1; /* bin. Semaphor fuer exkl. Schreiben */  
sem_r = 1; /* falls r=1 dann Lesen erlaubt, falls r=0 dann Lesen nicht erlaubt  
           (d. h. Schreibwunsch liegt vor) */
```

SYNCHRONISATION NEBENLÄUFIGER PROZESSE

Beispiel: zwei nebenläufige Leseprozesse R_1 und R_2 , o.B.d.A.: R_1 beginnt

R1	R2	gem. Variablen

start_read:		
DOWN(mutex_str)		mutex_str=0
DOWN(sem_r)		sem_r=0
DOWN(mutex_rc)		mutex_rc=0
-----	Zeitscheibe abgelaufen	-----
	start_read:	
	DOWN(mutex_str)	mutex_str=0 (R2)

man 7 sem_overview

A semaphore is an integer whose value is **never allowed to fall below zero**. Two operations can be performed on semaphores: increment the semaphore value by one (sem_post(3)); and decrement the semaphore value by one (sem_wait(3)). If the value of a semaphore is **currently zero**, then a sem_wait(3) operation will **block** until the value becomes greater than zero.

SYNCHRONISATION NEBENLÄUFIGER PROZESSE

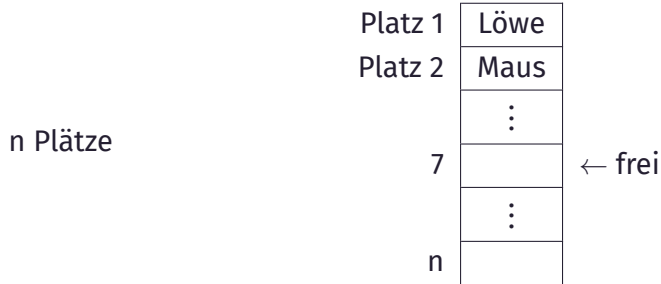
Beispiel: zwei nebenläufige Leseprozesse R_1 und R_2 , o.B.d.A.: R_1 beginnt

R1	R2	gem. Variablen
start_read:		
DOWN(mutex_str)		mutex_str=0
DOWN(sem_r)		sem_r=0
DOWN(mutex_rc)		mutex_rc=0
----- Zeitscheibe abgelaufen -----		
	start_read:	
	DOWN(mutex_str)	mutex_str=0 (R2)
----- R2 blockiert -----		
readcount++		readcount=1
DOWN(sem_w)		sem_w=0
UP(mutex_rc)		mutex_rc=1
----- Zeitscheibe abgelaufen -----		
UP(sem_r)		sem_r=1
UP(mutex_str)		mutex_str=0 ()
----- R2 bereit -----		
read(Daten)		
----- Zeitscheibe abgelaufen -----		

SYNCHRONISATION NEBENLÄUFIGER PROZESSE

		DOWN(sem_r)		sem_r=0
		DOWN(mutex_rc)		mutex_rc=0
		readcount++		readcount=2
	-----	Zeitscheibe abgelaufen	-----	
end_read:				
DOWN(mutex_rc)				mutex_rc=0 (R1)
	-----	R1 blockiert	-----	
		if(readcount==1)		(false)
		UP(mutex_rc)		mutex_rc=0 ()
	-----	R1 bereit	-----	
		UP(sem_r)		sem_r=1
	-----	Zeitscheibe abgelaufen	-----	
readcount--				readcount=1
if(readcount)				(false)
UP(mutex_rc)				mutex_rc=1
	-----	R1 fertig	-----	
		UP(mutex_str)		mutex_str=1
		read(Daten)		
		end_read:		
		...		
	-----	R2 fertig	-----	

ARCHE NOAH - GRUNDIDEE



frei := Variable, die die Nummer des nächsten zu belegenden Platzes enthält.

ARCHE NOAH

Semaphore functions in C:

Two operations can be performed on semaphores: **increment the semaphore value by one** (`sem_post(3)`); and **decrement the semaphore value by one** (`sem_wait(3)`). If the value of a semaphore is currently zero, then a `sem_wait(3)` operation will block until the value becomes greater than zero.

Race condition whilst creating a semaphore?:

The `sem_open(3)` function **creates a new named semaphore or opens an existing named semaphore**. After the semaphore has been opened, it can be operated on using `sem_post(3)` and `sem_wait(3)`. When a process has finished using the semaphore, it can use `sem_close(3)` to **close the semaphore**. When all processes have finished using the semaphore, it can be **removed from the system** using `sem_unlink(3)`.

ARCHE NOAH

The POSIX shared memory API allows processes to communicate information by sharing a region of memory. [...]

- `shm_open(3)` Create and open a new object, or open an existing object. [...]
- `ftruncate(2)` Set the size of the shared memory object. (A newly created shared memory object has a length of zero.)
- `mmap(2)` Map the shared memory object into the virtual address space of the calling process.
- `munmap(2)` Unmap the shared memory object from the virtual address space of the calling process.
- `shm_unlink(3)` Remove a shared memory object name.
- `close(2)` Close the file descriptor allocated by `shm_open(3)` when it is no longer needed.

Quelle:  `man 7 shm_overview`

UNTERSCHIED SEMAPHORE UND MUTEX

Vorlesung:

Binäre Semaphore werden auch als Mutex bezeichnet als Kurzform für Mutual Exclusion.



Mutex $\neq$ binäre Semaphore! **!**

Mutexe besitzen eine Besitzzuordnung (Ownership), Semaphore nicht!

UNTERSCHIED SEMAPHORE UND MUTEX

Semaphore:

```
sem_t s1; // this semaphore is initialized to 0
void client(){
    // do work
    sem_post(&s1);
}
void server(){
    sem_wait(&s1); // waits for client to finish work
    // do work
}
```

UNTERSCHIED SEMAPHORE UND MUTEX

Mutex:

Mutex Type	Robustness	Relock	Unlock When Not Owner
NORMAL	non-robust	deadlock	undefined behavior
NORMAL	robust	deadlock	error returned

- Robust Mutex ist ein Attribut welches dafür sorgt, dass ein Mutex **Fehler** liefert, wenn der Besitzerprozess **abstürzt**.
- Relock: Wenn ein Prozess versucht, einen Mutex zu sperren, den er bereits besitzt.
- Unlock When Not Owner: Wenn ein Prozess versucht, einen Mutex freizugeben, den er nicht besitzt.

OPTIMAL ALGORITHM

- Die Seite mit dem **größten Vorwärtsabstand** wird ausgelagert.

Vorwärtsabstand der **Seite** x zum **Zeitpunkt** t eines Programms mit **Laufzeit** T :

$$d(x, t) := \begin{cases} k, & \text{falls } x \notin \{s[t+1], \dots, s[t+k-1]\} \text{ und } s[t+k] = x \\ \infty, & \text{falls } x \notin \{s[t+1], \dots, s[T]\} \end{cases}$$

W	0	1	2	3	0	1	4	0	1	2	3	4
Rahmen: 0	0	0	0									
1	-	1	1									
2	-	-	2									
Seitenfehler	*	*	*									

- Achtung:** nur für Beurteilung anderer Verfahren interessant, denn $d(x, t)$ nicht vorher bekannt

LRU

- Least Recently Used

W	0	1	2	3	0	1	4	0	1	2	3	4
Rahmen: 0	0	0	0									
1	-	1	1									
2	-	-	2									
Seitenfehler	*	*	*									

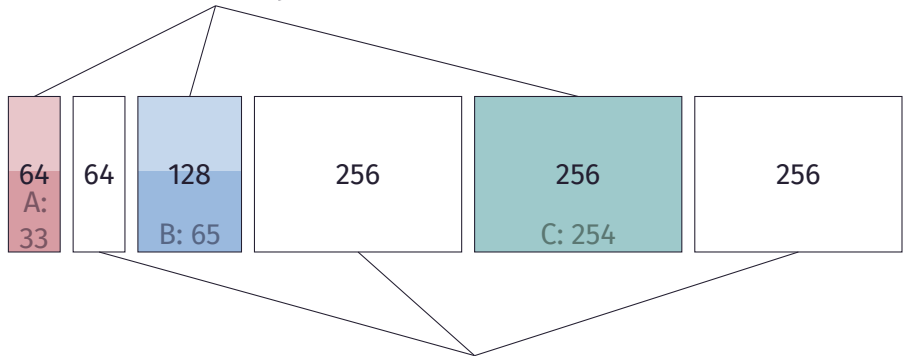
WORKING SET

- Working Set der **Größe** $T = 1$
- eine **beliebige Seite** die nicht im Working Set ist, wird ausgelagert:

W	0	1	2	3	0	1	4	0	1	2	3	4
Rahmen: 0	0	0	0									
1	-	1	1									
2	-	-	2									
Seitenfehler	*	*	*									

AUFGABE 2: SPEICHERVERWALTUNG

Interne Fragmentierung (Verschnitt)
innerhalb der Speicherblöcke



Externe Fragmentierung
des vorhandenen freien Speichers